

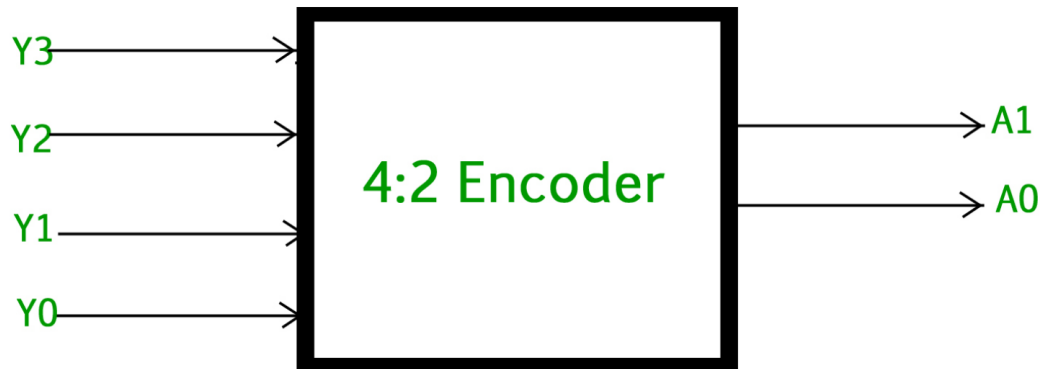
HW4

Implement a 4 to 1 encoder.

Circuit:

4 to 2 Encoder

The 4 to 2 Encoder consists of **four inputs** Y3, Y2, Y1 & Y0, and **two outputs** A1 & A0. At any time, only one of these 4 inputs can be '1' in order to get the respective binary code at the output. The figure below shows the logic symbol of the 4 to 2 encoder.



4 to 2 Encoder

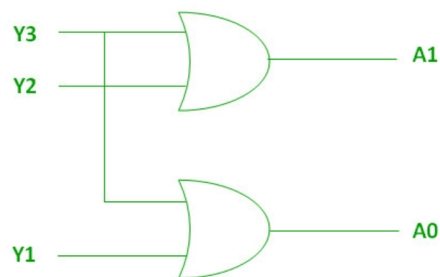
The Truth table of 4 to 2 encoders is as follows.

INPUTS				OUTPUTS	
Y3	Y2	Y1	Y0	A1	A0
0	0	0	1	0	0
0	0	1	0	0	1
0	1	0	0	1	0
1	0	0	0	1	1

Logical expression for A1 and A0:

$$\begin{aligned} A1 &= Y3 + Y2 \\ A0 &= Y3 + Y1 \end{aligned}$$

The above two Boolean functions A1 and A0 can be implemented using two input OR gates :



Implementation using OR Gate

Code:

BP	ln #	
	1	library IEEE;
	2	use IEEE.STD_LOGIC_1164.ALL;
	3	
	4	entity or2 is
	5	port(
	6	a, b : in STD_LOGIC;
	7	y : out STD_LOGIC
	8	);
	9	end entity;
	10	
	11	architecture rtl of or2 is
	12	begin
	13	y <= a or b;
	14	end architecture;

ln #	
1	library IEEE;
2	use IEEE.STD_LOGIC_1164.ALL;
3	
4	entity encoder4to2 is
5	port(
6	A : in STD_LOGIC_VECTOR(3 downto 0);
7	Y : out STD_LOGIC_VECTOR(1 downto 0)
8	);
9	end entity;
10	
11	architecture design of encoder4to2 is
12	component or2
13	port(
14	a, b : in STD_LOGIC;
15	y : out STD_LOGIC
16	);
17	end component;
18	
19	begin
20	o1: or2 port map(a => A(1), b => A(3), y => Y(0));
21	o2: or2 port map(a => A(2), b => A(3), y => Y(1));
22	end architecture;

Testbench:

ln	#
1	library IEEE;
2	use IEEE.STD_LOGIC_1164.ALL;
3	
4	entity tb_encoder4to2 is
5	end entity;
6	
7	architecture code of tb_encoder4to2 is
8	component encoder4to2
9	port(
10	A : in STD_LOGIC_VECTOR(3 downto 0);
11	Y : out STD_LOGIC_VECTOR(1 downto 0)
12	);
13	end component;
14	
15	signal A : STD_LOGIC_VECTOR(3 downto 0);
16	signal Y : STD_LOGIC_VECTOR(1 downto 0);
17	
18	begin
19	en: encoder4to2 port map(A => A, Y => Y);
20	
21	proc: process
22	begin
23	A <= "0001"; wait for 10 ns;
24	A <= "0010"; wait for 10 ns;
25	A <= "0100"; wait for 10 ns;
26	A <= "1000"; wait for 10 ns;
27	wait;
28	end process;
29	end architecture;

Wave:

wave - default		
Messages		
	/tb_encoder4to2/a	0001
	/tb_encoder4to2/y	00

wave - default		
Messages		
	/tb_encoder4to2/a	0010
	/tb_encoder4to2/y	01

wave - default		
Messages		
 /tb_encoder4to2/a	0100	0100
 /tb_encoder4to2/y	10	10

wave - default		
Messages		
  /tb_encoder4to2/a	1000	1000
  /tb_encoder4to2/y	11	11